

Descrição

Ao desenvolver uma aplicação utilizando o padrão MVP (Model-View-Presenter), é essencial compreender as responsabilidades de cada componente e a interação entre eles. Explique como o padrão MVP facilita a manutenção e a evolução de uma aplicação, utilizando um exemplo prático para ilustrar a interação entre o Model, a View e o Presenter.

luno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=s7YpA%252fmzt4vRw77bemWRuygng7CfVbzBos6yA8Ozeir8nL%.

25, 10:48

Portal do Aluno

Resposta Discursiva

O MVP já se destaca pela evolução em relação ao MVC por ter melhor separação da lógica da view. No MVP a View faz uma requisição ao presenter que estará responsável por controlar, processar os dados vindos da model e atualizar o view.

Nota Máxima

1,80

Nota

1,80

Comentário da Resposta

O MVP separa Model, View e Presenter, facilitando a organização, manutenção e evolução da aplicação. O Presenter controla a comunicação entre a View e o Model.

Descrição

Durante a transição para a arquitetura de microsserviços, é um tanto quanto comum que as empresas enfrentem desafios significativos relacionados à latência e à sobrecarga de chamadas de rede. Explique como a introdução de um *API Gateway* ajuda a mitigar esses problemas.

Resposta Discursiva

com a introdução de uma api gateway o sistema acaba ficando menos sobrecarregado e gera menos latência na rede, sem um gateway a rede fica muito pesada e lenta , por isso ocorre a sobrecarga e com a introdução o sistema consegue gerenciar melhor.

Nota Máxima

1,80

Nota

1,80

Comentário da Resposta

O API Gateway organiza e gerencia as requisições, reduzindo a sobrecarga da rede e melhorando o desempenho dos microsserviços.

Descrição

Uma forma muito adotada para o desenvolvimento de sistemas é a partir da definição de componentes. Observe a representação de um componente genérico a seguir:

Portal do Aluno



Escreva um parágrafo que justifique a utilização das componentes em desenvolvimento de sistemas. Cite duas vantagens em adotar essa estratégia e comente sobre as interfaces **requires** e **provides**.

Arquivo da Pergunta

Screenshot from 2022-09-12 23-42-38_20220912-2350.png (/ProvasOnline/Perguntas/Screenshot from 2022-09-12 23-42-38_20220912-2350.png)

Resposta Discursiva

A utilização de componentes no desenvolvimento de sistemas é muito impotante, por possuir diversos benefícios, dentre eles a escalabilidade, segurança, testabilidade, organização, eficiência e reutilização do mesmo componente em outros projetos semelhantes. a interface requires trata as requisições realizadas entre um componente e outro, enquanto a interface provides prove respostas as solicitações "requires" realizadas por um componente.

Nota Máxima

1,80

Nota

1,80

Requires - define os serviços de que o componente depende para executar.

Provides - define os serviços que ele oferece a outros componentes.

Vantagens: *provides* pode ser reutilizada, por não atender a uma função específica; *requires*, por deixar as dependências claras, facilita a relação com o sistema.

Descrição

A implementação de resiliência em arquiteturas distribuídas, especialmente em sistemas baseados em microservices, é fundamental para garantir a continuidade do serviço mesmo em cenários de falha. Considere um sistema de e-commerce que adota uma arquitetura de microservices para gerenciar diferentes funcionalidades, como pagamento, gerenciamento de estoque e processamento de pedidos.

no.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=gwY1QPAkP84f1oF2WZMgaMAISgaR0duCSHUF7jJSvQKMz2...

25, 08:07

Portal do Aluno

Descreva como técnicas de retry & fallback podem ser utilizadas para aumentar a resiliência do sistema, minimizando os impactos das falhas. Inclua exemplos práticos de como essas técnicas podem ser implementadas no contexto de microservices.

Resposta Discursiva

no contexto de microserviços, a técnica de retry é útil para recuperar-se de falhas que podem ocorrer em serviços isolados, já a técnica de fallback funciona como um reparo de emergência evitando a queda total de um serviço.

Nota Máxima

1,80

Nota

1,60

Comentário da Resposta

A resiliência em um sistema de microservices pode ser implementada através de técnicas de retry & fallback, que permitem ao sistema tentar novamente uma operação em caso de falha, ou recorrer a um método alternativo caso a operação falhe repetidamente. Por exemplo, no serviço de pagamento, se uma tentativa de processar um pagamento falhar devido a uma indisponibilidade temporária do serviço, o sistema pode automaticamente tentar novamente após um intervalo de tempo definido (retry). Se após várias tentativas o serviço ainda estiver indisponível, o sistema pode então acionar um fallback, como a utilização de um método de pagamento alternativo ou a colocação do pedido em uma fila para processamento posterior. A utilização dessas técnicas ajuda a garantir que o sistema continue a operar de maneira eficaz, mesmo quando ocorrem falhas em serviços individuais, aumentando a resiliência e minimizando os impactos na experiência do usuário.

Retry tenta novamente quando ocorre uma falha, e fallback usa uma alternativa quando o serviço principal não funciona. Assim, os microserviços ficam mais resilientes e evitam falhas em cadeia.

Descrição

No contexto do DevOps, a responsabilidade compartilhada entre as equipes de desenvolvimento e operações promove maior eficiência e confiabilidade na entrega de *software*. Um dos aspectos mais importantes dessa abordagem é o monitoramento contínuo, no qual as mesmas equipes que desenvolvem o software são responsáveis por sua operação em produção. Considere um sistema que exige alta disponibilidade, no qual falhas podem impactar diretamente a experiência do usuário. Descreva como a prática de responsabilidade compartilhada e monitoramento contínuo no DevOps contribui para a estabilidade e resiliência do sistema. Inclua exemplos práticos de como isso afeta a detecção e resolução de falhas em tempo real.

Resposta Discursiva

no devops todos que estão na equipe vão tratar o problema junto tratando com mais eficiência qualidade que caso ocorra algum erro para a solução do seja mais fácil já que está sendo tratado por toda equipe tendo mais compatibilidade de software

Nota Máxima

1,80

Nota

0,50

Comentário da Resposta

A responsabilidade compartilhada no DevOps garante que as equipes de desenvolvimento estejam diretamente envolvidas na operação do software em produção, o que contribui para uma maior atenção aos detalhes durante o processo de desenvolvimento. No caso de um sistema que exige alta disponibilidade, como uma plataforma de streaming, o monitoramento contínuo se torna essencial para identificar e resolver falhas antes que elas afetem significativamente os usuários. Com a prática de monitoramento ativo, os desenvolvedores têm visibilidade completa sobre o comportamento do sistema em produção, podendo agir de forma proativa para corrigir problemas. Por exemplo, se uma sobrecarga no servidor causar lentidão, o monitoramento detecta a anomalia e alerta a equipe, que pode escalar rapidamente os recursos, garantindo que o sistema continue funcionando sem interrupções. Essa abordagem colaborativa e contínua de monitoramento e ajuste em tempo real contribui para a resiliência do sistema, minimizando o impacto das falhas e

No DevOps, desenvolvimento e operações compartilham responsabilidades. O monitoramento contínuo permite identificar falhas rapidamente, corrigir problemas antes que afetem os usuários e manter o sistema estável e disponível.

Descrição

10/

<https://aluno.uvv.br/AgendamentoProva/VisualizarCorrecao>

O que é uma arquitetura de sistemas?

Resposta Discursiva

A arquitetura de sistemas é um subgrupo da Engenharia de Software responsável pelos componentes que irão compor o sistema e os requisitos não-funcionais. Ela é que irá definir sobre o reuso de componentes no desenvolvimento do software, visando a maior qualidade do código, segurança, confiabilidade e eficiência.

Nota Máxima

1,80

Nota

1,80

Comentário da Resposta

Arquitetura de sistemas é a área da Engenharia de Software que define os componentes e a estrutura de um sistema, buscando garantir qualidade, segurança, eficiência e confiabilidade.

Descrição

Cite duas formas de facilitarmos o reuso de código em nossos projetos.

Resposta Discursiva

Há diversas maneiras de facilitarmos o reuso de código em nossos projetos, destaco (1) a utilização de nomenclaturas mais genéricas, o que traz mais confiabilidade, credibilidade e referência para o nosso código, e (2) o uso de bibliotecas e frameworks pois, por serem soluções já utilizadas e testadas anteriormente, trazem também maior confiança, produtividade e desenvolvimento ágil para nossos projetos. Como bônus, gostaria de destacar o fato de que um código bem documentado é imprescindível, porque ajuda outros desenvolvedores no entendimento e nas tratativas de manutenção do código, por exemplo.

Nota Máxima

1,80

Nota

1,80

Comentário da Resposta

Usar nomenclaturas mais genéricas, facilitando a reutilização e manutenção.

Utilizar bibliotecas e frameworks, que já possuem códigos prontos, testados e confiáveis.

Descrição

Explique o que é o padrão arquitetural MVC

Resposta Discursiva

O padrão MVC realiza a comunicação entre o usuário e o programa, dividido em três camadas, Model(DADOS), Program(Lógica), e View(Interface) sendo a função de cada uma: Model: gerencia os sistema de dados e as operações, obtidos pelo view. Program: Manipula os dados armazenados em model. View: É a interface, recebe dados e comunica com o usuário.

Nota Máxima

1,80

Nota

1,80

Comentário da Resposta

MVC é um padrão que divide o sistema em 3 partes:

- **Model:** cuida dos dados.
- **View:** é a interface com o usuário.
- **Controller:** faz a ligação entre o usuário, a View e o Model.

Descrição

É inegável que analisar tarefas comuns que os usuários realizarão no sistema pode ajudar bastante a identificar áreas onde a eficiência pode ser aprimorada, resultando em uma experiência de usuário mais fluida e satisfatória, em que a produtividade é maximizada e a frustração minimizada. Explique como a otimização de fluxos de trabalho e a redução de cliques desnecessários podem melhorar a eficiência de uma interface, usando exemplos práticos.

Resposta Discursiva

Nota Máxima

1,80

Nota

0,00

Comentário da Resposta

A otimização de fluxos de trabalho e a redução de cliques desnecessários são fundamentais para melhorar a eficiência de uma interface. Isso pode ser conseguido analisando as tarefas comuns que os usuários realizam e identificando áreas onde o processo pode ser simplificado. Por exemplo, em um software de edição de vídeo, permitir que os usuários arrastem e soltem efeitos diretamente na linha do tempo em vez de navegar por múltiplos menus reduz o número de cliques necessários para completar uma tarefa, economizando tempo e esforço. Teclas de atalho configuráveis também melhoram a eficiência, permitindo que usuários experientes realizem ações comuns com um único pressionar de tecla, como cortar ou renderizar vídeos. Esses exemplos mostram como o design centrado na eficiência pode levar a uma experiência de usuário mais fluida e satisfatória, aumentando a produtividade e diminuindo a frustração.

Analisar as tarefas mais comuns dos usuários permite identificar e eliminar etapas desnecessárias, reduzindo cliques e tornando os fluxos de trabalho mais rápidos e eficientes. Por exemplo, usar atalhos, menus simplificados ou preencher informações automaticamente pode facilitar a realização das tarefas, aumentando a produtividade e a satisfação do usuário.

Descrição

Design Patterns ou padrões de projetos são soluções generalistas para problemas recorrentes durante o desenvolvimento de um software. Observe as seguintes afirmações sobre Design Patterns:

- I - Permite a reutilização de módulos do sistema em outros sistemas
- II - Reduz simultaneamente a complexidade no código e a produtividade
- III - Compromete a produtividade ao tratar de problemas inovadores
- IV - Diminui o tempo de desenvolvimento de um projeto

É correto o que se afirma em:



A

Afirmativas II e III

☐

Portal do Aluno

B

Afirmativas I e IV



C

Afirmativas III e IV

☐

D

Afirmativas I e II

☐

E

Todas as afirmativas

☐

Descrição

A cultura DevOps é construída com base em colaboração, responsabilidade compartilhada e feedback contínuo entre as equipes de desenvolvimento e operações. Essa abordagem visa melhorar a eficiência, promover a inovação e garantir uma entrega contínua e de alta qualidade do software.

Fonte: criação do autor.

Marque a alternativa correta, entre as que estão a seguir, sobre os benefícios da comunicação assíncrona entre serviços:

**A**

A responsabilidade compartilhada promove maior cuidado com o código, pois os desenvolvedores são também responsáveis pela operação em produção.

**B**

A responsabilidade compartilhada no DevOps significa que os operadores de sistemas assumem total responsabilidade pela correção de falhas no código.

**C**

A responsabilidade compartilhada elimina a necessidade de colaboração entre equipes, já que todos assumem a mesma função.

**D**

A responsabilidade compartilhada permite que as equipes de desenvolvimento se concentrem apenas na criação de novas funcionalidades, sem se preocupar com a manutenção do código.

**E**

A responsabilidade compartilhada no DevOps reduz a necessidade de automação, uma vez que a equipe de operações tem maior controle sobre o processo de implantação e monitoramento.

Descrição

Uma equipe de DevOps está implementando contêineres e orquestração com Docker e Kubernetes para um grande projeto de software em produção. O objetivo principal é garantir a escalabilidade da aplicação e a sua capacidade de recuperação automática em caso de falhas. A equipe precisa configurar o Kubernetes para que a solução produza o efeito que dela se espera.

Fonte: criação do autor.

Considerando esse cenário, marque a alternativa que contém a afirmativa correta sobre o Kubernetes:

**A**

Oferece escalabilidade automática e recuperação de contêineres com falhas, garantindo alta disponibilidade.

**B**

Não suporta o escalonamento automático, necessitando que todos os contêineres sejam manualmente gerenciados.

**C**

É limitado à gestão de contêineres em um único nó, sem suporte para clusters distribuídos.

**D**

Carece de mecanismos de recuperação automática em caso de falhas nos contêineres.

**E**

Impossibilita o balanceamento de carga entre contêineres, exigindo uma abordagem manual para distribuir o tráfego.

Descrição

A comunicação entre serviços em uma arquitetura distribuída é essencial para garantir que as diversas partes de um sistema possam interagir e funcionar de maneira coesa. Essa comunicação pode ocorrer de forma síncrona ou assíncrona, e a escolha do método depende das necessidades específicas do sistema e das características dos serviços envolvidos.

Fonte: criação do autor.

Considere as afirmações a seguir sobre mensagens e comunicações entre serviços. Marque V para verdadeiro e F para falso.

() O uso de filas de mensagens, como as oferecidas pelo RabbitMQ e Apache Kafka, permite que a comunicação entre serviços ocorra de maneira assíncrona, garantindo que mensagens sejam entregues mesmo em caso de falhas temporárias.

() A comunicação síncrona, como aquela realizada via chamadas HTTP, é preferível quando a resposta imediata é crítica para a operação do sistema, apesar de poder introduzir bloqueios em caso de indisponibilidade do serviço receptor.

() A comunicação assíncrona permite que os serviços sejam mais desacoplados, aumentando a resiliência do sistema ao não exigir que todos os serviços estejam disponíveis simultaneamente.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras e falsas



A

F - F - V

☐

B

V - F - F

☐

C

V - V - F

☐

D

F - F - F

☐

E

V - V - V

☒

Descrição

No DevOps, a colaboração contínua entre equipes de desenvolvimento e operações é essencial para o sucesso da metodologia. A responsabilidade compartilhada e o *feedback* constante promovem a rápida identificação de problemas e garantem que o *software* atenda aos padrões de qualidade e segurança.

Fonte: criação do autor.

Considere as afirmações a seguir sobre colaboração e responsabilidade no DevOps. Marque V para verdadeiro e F para falso.

- () A responsabilidade compartilhada no DevOps significa que a mesma equipe que desenvolve o *software* é responsável por sua operação em produção, promovendo maior cuidado com o código.
- () No DevOps, o *feedback* contínuo só é relevante após a fase de desenvolvimento, quando o *software* já está em produção.
- () A comunicação entre as equipes de desenvolvimento e operações é um dos pilares do DevOps, garantindo uma visão unificada do ciclo de vida do *software*.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras e falsas.



A

F - V - V

☐

B

V - F - F.

☐

C

V - V - F

☐

D

F - F - V

☐

E

V - F - V

☒

Descrição

Uma startup de tecnologia está migrando sua infraestrutura para um modelo de Infraestrutura como Código (IaC) para aumentar a eficiência e reduzir erros manuais. O CTO da empresa solicitou que a equipe de DevOps configure todo o ambiente de produção usando ferramentas de IaC, permitindo que a infraestrutura seja versionada e replicada facilmente. A equipe está considerando o uso de Terraform para essa tarefa, mas precisa garantir que as configurações sejam consistentes entre os diferentes ambientes de desenvolvimento, teste e produção.

Fonte: Criação do autor.

Considerando esse cenário, marque a alternativa que contém a afirmativa correta sobre o Terraform:



A

Permite o provisionamento de infraestrutura em múltiplos provedores de nuvem, mantendo consistência entre ambientes.



B

Não suporta controle de versão, necessitando que as configurações sejam gerenciadas manualmente.



C

Requer que as configurações sejam feitas diretamente em cada servidor, sem automação de provisionamento.



D

É limitado a provisionar infraestrutura apenas em ambientes locais, sem suporte para nuvens públicas.



E

Inibe a replicação de infraestrutura entre ambientes de teste e produção de maneira automatizada.

Descrição

No DevOps, o *feedback* contínuo desempenha um papel crucial para a melhoria constante dos processos de desenvolvimento e operação. Essa, aliás, acaba se revelando uma das maiores razões da grande popularidade da abordagem DevOps.

Fonte: Criação do autor.

Marque a alternativa correta, entre as que estão a seguir, sobre os benefícios do *feedback* contínuo no DevOps:



A

O *feedback* contínuo ajuda a identificar problemas rapidamente, promovendo ajustes rápidos e melhorias contínuas no *software*.



B

O *feedback* contínuo só é relevante após a implementação do *software* em produção, quando se pode medir o desempenho.



C

No DevOps, o *feedback* contínuo substitui a necessidade de testes automatizados, pois os problemas são identificados ao longo do tempo.



D

O *feedback* contínuo promove uma cultura na qual apenas a equipe de operações é responsável por reportar falhas e sugerir melhorias.



E

O *feedback* contínuo é utilizado apenas durante o desenvolvimento inicial do código, pois após a implantação o *software* já está otimizado.

Descrição

Ao desenvolver uma aplicação web usando o padrão MVC, o primeiro passo que o profissional responsável precisa seguir é o de identificar com muita clareza quais são as responsabilidades de cada componente – Model, View e Controller.

Fonte: criação do autor.

Analise as assertivas a seguir.

I. O Model no padrão MVC representa a lógica de negócios e os dados da aplicação.

II. A View no padrão MVC é responsável por validar as entradas do usuário.

III. O Controller no padrão MVC atua como intermediário entre a View e o Model, processando entradas e atualizando os dados.

Marque a alternativa a seguir que está correta em ao indicar quais das assertivas acima são verdadeiras.

A

Somente a I.



B

Somente I e III.



C

Somente a III.



D

Somente I e II.



E

Somente a II e III.



Nota

0,00



Os usuários esperam que os elementos de interface funcionem de maneira previsível, e isso pode ser alcançado através da aplicação consistente de padrões de design em todo o sistema. A consistência não se refere apenas à aparência visual, mas também ao comportamento funcional dos elementos, como botões, menus e formulários.

Fonte: criação do autor.

Sobre a importância da consistência no design de interfaces, marque a alternativa correta a seguir:



A
Melhora a eficiência do uso ao reduzir a curva de aprendizado.



B
É nociva para a aparência visual dos elementos.



C
Pouco afeta a confiança dos usuários nas interfaces.



D
Aumenta a necessidade de reaprendizagem em novas telas.



E
Torna os elementos de interface menos previsíveis.

Ao se ponderar acerca da seleção de um padrão arquitetural como as alternativas oferecidas pelos padrões MVC, MVP e MVVM, é preciso lembrar que a complexidade da aplicação também acaba por influenciar a escolha do padrão mais apropriado.

Fonte: criação do autor.

Sobre a influência da complexidade da aplicação na escolha do padrão arquitetural, marque a alternativa correta.



A

O MVC é ideal para interfaces complexas com data binding.

☐

B

O MVP é adequado para lógica de apresentação complexa.



C

O MVVM é mais adequado para interfaces simples.

☐

D

O MVC é complexo de implementar para interfaces simples.

☐

E

O MVP reduz o código boilerplate em interfaces complexas.

☐

Descrição

Ao desenvolver uma aplicação web usando o padrão MVC, o primeiro passo que o profissional responsável precisa seguir é o de identificar com muita clareza quais são as responsabilidades de cada componente – Model, View e Controller.

Fonte: criação do autor.

Analise as assertivas a seguir.

I. O Model no padrão MVC representa a lógica de negócios e os dados da aplicação.

II. A View no padrão MVC é responsável por validar as entradas do usuário.

III. O Controller no padrão MVC atua como intermediário entre a View e o Model, processando entradas e atualizando os dados.

Marque a alternativa a seguir que está correta em ao indicar quais das assertivas acima são verdadeiras.



A

Somente a I.

☐

B

Somente I e III.



C

Somente a III.

☐

D

Somente I e II.

☐

E

Somente a II e III.

☐

Descrição

Os protocolos de comunicação e os formatos de serialização são componentes críticos na troca de dados entre serviços em arquiteturas distribuídas. Protocolos como HTTP e HTTPS são amplamente utilizados, enquanto formatos de serialização como JSON e XML são preferidos por sua compatibilidade e legibilidade.

Fonte: criação do autor.

Marque a alternativa correta sobre o uso de protocolos e formatos de serialização entre serviços:



A

HTTPS é a versão segura do protocolo HTTP, utilizando criptografia para proteger a comunicação entre serviços.



B

O protocolo HTTP é ideal para comunicação segura entre serviços em sistemas distribuídos.



C

O formato XML é sempre preferido em APIs modernas devido à sua simplicidade e menor tamanho em comparação com JSON.



D

A serialização em formato binário, como Protocol Buffers, é utilizada principalmente por ser mais legível que JSON e XML.



E

A comunicação via HTTP é inadequada para sistemas que exigem resposta em tempo real.



Descrição

Ao desenvolver uma aplicação web usando o padrão MVC, o primeiro passo que o profissional responsável precisa seguir é o de identificar com muita clareza quais são as responsabilidades de cada componente – Model, View e Controller.

Fonte: criação do autor.

Analise as assertivas a seguir.

I. O Model no padrão MVC representa a lógica de negócios e os dados da aplicação.

II. A View no padrão MVC é responsável por validar as entradas do usuário.

III. O Controller no padrão MVC atua como intermediário entre a View e o Model, processando entradas e atualizando os dados.

Marque a alternativa a seguir que está correta em ao indicar quais das assertivas acima são verdadeiras.



A

Somente a I.

☐

B

Somente I e III.



C

Somente a III.

☐

D

Somente I e II.

☐

E

Somente a II e III.

A disponibilidade de um sistema é uma medida crítica que reflete o tempo em que o sistema está operacional e acessível aos usuários. Em arquiteturas de sistemas, garantir alta disponibilidade envolve a implementação de diversas estratégias e práticas que minimizam o tempo de inatividade.

Fonte: criação do autor.

Marque a alternativa a seguir que está correta sobre as práticas que garantem alta disponibilidade:



A

A replicação de dados em tempo real minimiza o impacto de falhas.



B

A manutenção corretiva é suficiente para garantir alta disponibilidade.



C

O uso de servidores de failover é dispensável em sistemas resilientes.



D

A alta disponibilidade é garantida sem necessidade de monitoramento.



E

Backups regulares substituem a necessidade de sistemas de failover.



Descrição

Garantir o desempenho de um sistema de software é crucial para atender às expectativas dos usuários e manter a competitividade no mercado. Em arquiteturas de sistemas, diversos fatores influenciam o desempenho, incluindo tempo de resposta, taxa de transferência e capacidade de processar grandes volumes de dados.



Fonte: criação do autor.

Marque a alternativa a seguir que está correta sobre as práticas de otimização de desempenho:

- A** O uso de caching melhora o tempo de resposta em sistemas web. ☒
- B** O balanceamento de carga reduz a latência para dados em repouso. ☐
- C** ☐

A computação in-memory é ideal para reduzir o tráfego de rede.



- D** A otimização de código é desnecessária em arquiteturas distribuídas. ☐
- E** O tempo de resposta não é afetado pelo número de requisições simultâneas. ☐

Descrição

Implementar efetivamente uma arquitetura de sistemas previamente selecionada pode ser entendido tanto como uma arte quanto como uma ciência: arte porque envolve criatividade e visão, ciência porque requer conhecimento técnico e metodologia.



Fonte: criação do autor.

Sobre a implementação de uma arquitetura de sistemas em uma organização, marque a alternativa correta a seguir:

- A** Pode ser realizada sem planejamento prévio. ☐
- B** Dispensa a análise de impacto nos negócios. ☐
- C** É exclusivamente uma responsabilidade de TI. ☐

- D** É algo alheio a questões de segurança dos dados. ☐
- E** Requer planejamento estratégico e técnico. ☒

Nota 0,60

Descrição

A resiliência em sistemas de software é fundamental para garantir que os serviços permaneçam operacionais mesmo em face de falhas ou condições adversas. A implementação de práticas de resiliência envolve diversas estratégias e ferramentas que permitem que o sistema continue a funcionar, ou se recupere rapidamente, mesmo quando partes dele falham.



Fonte: criação do autor.

Considere as afirmações a seguir sobre a implementação de resiliência em sistemas de software. Marque V para verdadeiro e F para falso.

- () Arquiteturas baseadas em microservices permitem maior resiliência ao isolar falhas em serviços individuais, evitando que elas afetem todo o sistema.
- () A replicação ativa-ativa, na qual múltiplas instâncias do sistema operam simultaneamente, diminui a resiliência, pois aumenta o risco de falhas simultâneas.
- () Ferramentas de monitoramento como Prometheus e Grafana são utilizadas para identificar problemas em tempo real e melhorar a resiliência do sistema.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras e falsas

A F - F - V

☐

B V - F - F.

☐

C V - V - F

☐

D F - V - V

☐

E V - F - V



Nota 0,60

Descrição

Uma equipe de desenvolvimento de software em uma empresa de tecnologia, visando melhorar a eficiência e a qualidade de suas entregas, decidiu implementar práticas de DevOps, com ênfase em Integração Contínua (CI) e Entrega Contínua (CD). Durante a revisão dos processos, foi destacada a importância de garantir que todo o código, após ser validado por meio de testes automatizados, pudesse ser liberado para produção de forma rápida e segura, sem comprometer a estabilidade dos sistemas em uso. O gerente de TI solicitou que a equipe validasse as práticas e garantisse que todas as etapas fossem automatizadas de acordo com os melhores padrões de DevOps, sem abrir mão da qualidade.



Fonte: criação do autor.

Sobre a prática de Entrega Contínua (CD) no DevOps, veja as assertivas a seguir e marque V (verdadeiro) ou F (falso) para cada uma delas:

() O objetivo da Entrega Contínua é garantir que o software esteja sempre em estado de "pronto para produção", facilitando a entrega de novas funcionalidades com segurança e agilidade.

() A Entrega Contínua automatiza todo o ciclo de desenvolvimento, desde a codificação inicial até o envio ao ambiente de produção, sem necessidade de validação manual em nenhum estágio.

() Um pipeline de CD abrange as etapas de construção, testes automatizados e implantação em ambientes de produção ou pré-produção,

minimizando o risco de erros humanos.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras e falsas

A F - V - V

☐

B V - F - F

☐

C V - V - F

☐

D F - F - V

☐

E V - F - V



Descrição	Nos sistemas informatizados atuais, a necessidade de integração eficiente, comunicação em tempo real e escalabilidade tem levado à recorrência do uso de APIs, que se tornaram essenciais para conectar diferentes plataformas e serviços, facilitando a interoperabilidade e a inovação contínua em ambientes tecnológicos complexos.	
------------------	--	---

Fonte: criação do autor.

Considere as seguintes afirmações sobre a integração de sistemas e APIs:

I. A arquitetura REST é conhecida por sua escalabilidade e independência de plataforma, sendo amplamente adotada em aplicações web modernas.

II. O uso de APIs facilita a comunicação entre sistemas, mas requer a implementação de mecanismos de segurança robustos para proteger dados sensíveis durante a transferência.

III. Protocolos como gRPC e SOAP são alternativas ao REST, oferecendo suporte a cenários de alta performance e transações seguras, respectivamente.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras.

- | | | |
|----------|-------------------|-------------------------------------|
| A | Somente I e II. | ☐ |
| B | Somente I e III. | ☐ |
| C | Somente a III. | ☐ |
| D | I, II e III. | ☒ |
| E | Somente II e III. | ☐ |

Descrição

A aplicação dos padrões arquiteturais MVC, MVP e MVVM varia de acordo com o contexto do projeto, a complexidade da aplicação e as tecnologias utilizadas. O padrão MVC (Model-View-Controller) é amplamente utilizado em algumas determinadas aplicações por suas distintas características.

Fonte: criação do autor.

Sobre a aplicação do padrão MVC, marque a alternativa correta a seguir.



A

Facilita colaboração entre equipes de front-end e back-end.



B

É raramente utilizado em aplicações web modernas.



C

Dificulta a manutenção independente dos componentes.

uno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252ft3psgaMAISsgaQC8A/veCDxy4EasgaMAI...

25, 08:12

Portal do Aluno



D

É incompatível com frameworks populares.



E

É ideal para aplicações desktop simples.



Descrição

Uma startup de tecnologia está migrando sua infraestrutura para um modelo de infraestrutura como Código (IaC) para aumentar a eficiência e reduzir erros manuais. O CTO da empresa solicitou que a equipe de DevOps configure todo o ambiente de produção usando ferramentas de IaC, permitindo que a infraestrutura seja versionada e replicada facilmente. A equipe está considerando o uso de Terraform para essa tarefa, mas precisa garantir que as configurações sejam consistentes entre os diferentes ambientes de desenvolvimento, teste e produção.

Fonte: Criação do autor.

Considerando esse cenário, marque a alternativa que contém a afirmativa correta sobre o Terraform:



A

Permite o provisionamento de infraestrutura em múltiplos provedores de nuvem, mantendo consistência entre ambientes.



B

Não suporta controle de versão, necessitando que as configurações sejam gerenciadas manualmente.



C

Requer que as configurações sejam feitas diretamente em cada servidor, sem automação de provisionamento.



D

É limitado a provisionar infraestrutura apenas em ambientes locais, sem suporte para nuvens públicas.



E

uno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252ft3psqaMAISsgaQC8AveCDxy4EasgaMAI...

Descrição

A implementação de projetos em Python utilizando as arquiteturas SOA, microsserviços e orientada a eventos envolve diversas etapas e considerações técnicas, que todo bom profissional envolto em projetos de sistemas informatizados precisa dominar.

Fonte: criação do autor.

Considere as afirmações a seguir sobre a implementação de projetos em Python utilizando essas arquiteturas. Marque V para verdadeiro e F para falso.

- () O Flask é comumente utilizado para desenvolver APIs REST em uma arquitetura de microsserviços.
- () Em SOA, cada serviço deve compartilhar a mesma base de dados para manter a consistência.
- () O uso de bibliotecas de mensageria como Kafka é comum em arquiteturas orientadas a eventos para comunicação assíncrona.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras e falsas



A

F - F - V

☐

B

V - F - F.

☐

C

V - V - F

☐

D

F - V - V

no.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252f3psqaMAISsgaQC8AVeCDxy4EasgaMAI..

5, 08:12

Portal do Aluno



E

V - F - V



Descrição

A manutenção de sistemas é crucial para a sustentabilidade a longo prazo de aplicações de *software*. Quem ignora isso acaba enfrentando problemas de obsolescência, vulnerabilidades de segurança e dificuldades crescentes para acompanhar as demandas do mercado e as evoluções tecnológicas, comprometendo a competitividade e a eficácia do negócio.

Fonte: criação do autor.

Analise as assertivas a seguir.

I. Arquiteturas complexas aumentam os custos de manutenção.

II. A manutenção de *software* é irrelevante para arquiteturas *serverless*.

III. Estratégias proativas de manutenção reduzem interrupções do sistema.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras.



A

Somente a I.

☐

B

Somente I e III.



C

Somente a III.

☐

D

Somente I e II.

uno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252ft3psqaMAISsgaQC8AVeCDxy4EasgaMAI...

25, 08:12

Portal do Aluno

☐

E

Somente a II.

[

Descrição

A comunicação assíncrona é uma prática comum em arquiteturas de microserviços, permitindo que serviços se comuniquem de maneira eficiente sem precisar esperar uma resposta imediata. Esse método de comunicação é essencial para manter a resiliência e escalabilidade dos sistemas distribuídos.

Fonte: criação do autor.

Marque a alternativa correta sobre os benefícios da comunicação assíncrona entre serviços:



A

A comunicação assíncrona facilita a integração entre serviços, permitindo que a disponibilidade de um serviço não impacte diretamente outro serviço.



B

A comunicação assíncrona elimina a necessidade de filas de mensagens em sistemas distribuídos.



C

A comunicação assíncrona é ideal para transações financeiras que requerem confirmação imediata.



D

A comunicação assíncrona não pode ser utilizada em sistemas que precisam de alta disponibilidade.



E

A comunicação assíncrona exige que todos os serviços estejam disponíveis simultaneamente para funcionar corretamente.

Descrição

No processo de design, implementação e implantação de serviços, diferentes tecnologias e abordagens são utilizadas. Saber quais selecionar e explorar é, em última instância, uma questão de alinhar as necessidades do projeto com as melhores práticas e ferramentas disponíveis, garantindo eficiência, segurança e escalabilidade.

Fonte: criação do autor.

Marque a alternativa correta sobre os desafios enfrentados neste processo:



A

A orquestração e a monitoração são simplificadas.

☐

B

A gestão de dados é simples devido à centralização.

☐

C

A comunicação entre serviços pode introduzir latência.



D

Garantem automaticamente consistência de dados.

☐

E

Reduzem a necessidade de autenticação robusta.



Descrição

Abundam desafios no dia a dia típico de um arquiteto de sistemas, que, não raro, se vê equilibrando demandas concorrentes de diversos *stakeholders*, desde requisitos técnicos até restrições orçamentárias, buscando alcançar soluções eficientes e alinhadas com os objetivos do negócio.
Fonte: criação do autor.

Sobre os desafios na implementação e manutenção de uma arquitetura de sistemas, marque a alternativa correta a seguir:

**A**

Todos os sistemas são inerentemente simples de integrar.

☐**B**

Inexistem desafios no caso de arquiteturas monolíticas atualmente.

☐**C**

A escalabilidade é automaticamente garantida pelo *blockchain*.

☐**D**

A comunicação entre serviços distribuídos pode ser complexa.

**E**

Os custos são irrelevantes na escolha da arquitetura.

☐

Descrição

O MVC é um padrão:



A

de arquitetura de aplicações que visa separar a "lógica da aplicação" da "interface do usuário" e do "fluxo da aplicação".



B

de objeto que executa uma operação quando ele recebe uma solicitação (mensagem) correspondente de um outro objeto.



C

de desenvolvimento cuja finalidade é definir uma interface na mesma camada da "lógica da aplicação"



D

de operação dirigida para uma classe e não para um objeto individual.



E

que garante que uma lógica de negócios possa ser acessada e visualizada por uma única interface.

Descrição

Uma equipe de DevOps está implementando contêineres e orquestração com Docker e Kubernetes para um grande projeto de software em produção. O objetivo principal é garantir a escalabilidade da aplicação e a sua capacidade de recuperação automática em caso de falhas. A equipe precisa configurar o Kubernetes para que a solução produza o efeito que dela se espera.

Fonte: criação do autor.

Considerando esse cenário, marque a alternativa que contém a afirmativa correta sobre o Kubernetes:



A

Oferece escalabilidade automática e recuperação de contêineres com falhas, garantindo alta disponibilidade.



B

Não suporta o escalonamento automático, necessitando que todos os contêineres sejam manualmente gerenciados.

C

É limitado à gestão de contêineres em um único nó, sem suporte para clusters distribuídos.



D

Carece de mecanismos de recuperação automática em caso de falhas nos contêineres.



E

Impossibilita o balanceamento de carga entre contêineres, exigindo uma abordagem manual para distribuir o tráfego.



É justo reconhecer que o célebre autor canadense Thomas Erl é um dos mais fervorosos defensores da Arquitetura Orientada a Serviços (SOA) – e, certamente, toda a paixão que ele nutre por essa vertente se deu a partir do claro entendimento das principais características conceituais envolvidas.

Fonte: criação do autor.

Marque a alternativa correta sobre as vantagens da SOA:



A

Facilita a escalabilidade e a modularidade.



B

Garante consistência de dados em tempo real.

☐

C

Depende de uma base de dados monolítica.

☐

D

uno.uv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252ft3psgaMAISsgaQC8AVeCDxy4EasgaMAI..

25, 18:23

Portal do Aluno

Evita a necessidade de qualquer forma de monitoramento.

☐

E

Simplifica a integração de serviços legados.

O paradigma da arquitetura orientada a serviços (SOA - Service-Oriented Architecture) foi criado com foco em prover serviços fracamente acoplados e coesos como resposta à demanda de integração tecnológica global com diversidade em tecnologias, plataformas e regras de negócio.

Apesar de ser utilizada em diversas aplicações, nos dias atuais esse paradigma ainda não é altamente difundido, isso se deve principalmente porque



A

não existe interesse genuíno das grandes empresas tecnológicas em adotar a SOA e sim em manter estrutura de sistemas orientado a objetos.

☐

B

esse paradigma levaria a um alto custo, pois pressupõe alterar a forma de modelar algo que já está pronto e funcional.



C

não existem vantagens concretas em sua adoção, sendo um paradigma específico para sistemas ineficientes.

☐

D

é muito vantajoso propor uma nova arquitetura para algo que já está em pleno funcionamento.

☐

ino.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252ft3psgaMAISsgaQC8AVeCDxy4EasgaMAI..

E

essa tecnologia está em ascensão, sendo ainda muito recente para que ocorra sua completa adoção.

Descrição

Métodos ágeis encontram um formidável território de aplicação junto ao design de interface de sistemas informatizados, até mesmo por uma especial capacidade de contribuição no que diz respeito à prevenção dos erros mais corriqueiros dessa atividade.

Fonte: criação do autor.

Considere as afirmações a seguir sobre metodologias ágeis no design de interfaces. Marque V para verdadeiro e F para falso.

- () Scrum organiza o trabalho em sprints curtos, geralmente de duas a quatro semanas.
- () Kanban descarta o fluxo de trabalho e a visualização das tarefas.
- () Ambas as metodologias, Scrum e Kanban, ajudam a manter a flexibilidade e a eficiência.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras e falsas



A

V - F - V



B

V - F - F



C

V - V - F



D

F - V - V

uno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252ft3psgaMAISsgaQC8AVeCDxy4EasgaMAI...

25, 18:23

Portal do Aluno



E

F - F - V

Descrição

No DevOps, a prática de monitoramento contínuo desempenha um papel de suma importância para garantir a estabilidade e o desempenho de aplicações em produção. Além disso, o uso de ferramentas como Prometheus e Grafana permite a visualização em tempo real das métricas, auxiliando no diagnóstico de falhas e na melhoria da performance.

Fonte: criação do autor.

A partir disso, veja as asserções abaixo:

I. O monitoramento contínuo em DevOps ajuda a detectar falhas antes que impactem os usuários.

II. Ferramentas como Prometheus são utilizadas apenas para monitorar redes, sem integração com aplicações.

III. O Grafana permite a criação de dashboards personalizados para acompanhar métricas de desempenho.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras.



A

Somente I e II.

☐

B

Somente I e III.



C

Somente a III.

☐

D

I, II e III.

ino.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno?parametros=%252fGjGPZQ7P%252ft3psgaMAISsgaQC8AVeCDxy4EasgaMAI..

25, 18:23

Portal do Aluno



E

Somente a II e III.

☐

Descrição

As tendências atuais na arquitetura de *software* refletem a constante evolução e adaptação às novas demandas tecnológicas e de mercado.

Fonte: criação do autor.

Considere as afirmações a seguir sobre as tendências em arquitetura de *software*. Marque V para verdadeiro e F para falso.

- () Arquiteturas nativas da nuvem ignoram a escalabilidade automática.
- () Arquiteturas imutáveis reduzem inconsistências de configuração.
- () Arquiteturas híbridas combinam infraestrutura em nuvem e sistemas *on-premises*.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras e falsas



A

325, 18:08

Portal do Aluno

☐

B

V - F - F

☐

C

V - V - F

☐

D

F - V - V

☐

E

F - F - V



Descrição

No DevOps, a prática de integração contínua (CI) e entrega contínua (CD) é fundamental para a automação e a eficiência na entrega de *software*. Além disso, o *feedback* contínuo entre as equipes garante que melhorias sejam implementadas rapidamente.

Fonte: criação do autor.

A partir disso, veja as asserções abaixo:

I. A integração contínua (CI) permite testes automáticos do código.

II. A entrega contínua (CD) dispensa revisões de código antes da implantação.

III. O *feedback* contínuo no DevOps ajuda a identificar e corrigir problemas rapidamente.

Marque a alternativa correta, entre as que estão a seguir, em indicar quais das assertivas acima são verdadeiras.



A

Somente I e II.

☐

B

Somente I e III.



C

Somente a III.

☐

D

I, II e III.

☐

E

Somente a II e III.

Descrição

A segurança em sistemas de software é uma qualidade básica a ser considerada, e isso tem suas devidas justificativas – afinal, segurança é estritamente fundamental para proteger apropriadamente dados e serviços contra ameaças.

Fonte: criação do autor.

Análise as assertivas a seguir.

I. Tokens JWT (JSON Web Tokens) são utilizados para autenticação e autorização seguras, permitindo um controle granular de acesso.

II. O protocolo OAuth expõe as credenciais do usuário, tornando-o menos seguro que outras soluções de autenticação.

III. A criptografia dos dados, tanto em trânsito quanto em repouso, é essencial para proteger as informações contra interceptações e acessos não autorizados.

Marque a alternativa a seguir que está correta em indicar quais das assertivas acima são verdadeiras.



A

Somente a I.

☐

B

Somente I e III.



C

Somente a III.

☐

D

Somente I e II.

☐

E

Somente II e III.

Descrição

Uma vez concluída a definição dos requisitos, entra-se na fase de design conceitual. Tal fase é notabilizada por ser o momento em que são criados wireframes e protótipos de baixa fidelidade que delineiam a estrutura básica da interface.

Fonte: criação do autor.

Sobre as etapas do design de interface, marque a alternativa correta a seguir:



A

Wireframes incluem detalhes completos e finalísticos de design visual.



B

Wireframes são esboços simplificados que mostram a estrutura de uma página.



C

Prototipagem de baixa fidelidade é uma instância não iterativa por definição.



D

A fase de desenvolvimento ignora o design detalhado.



E

Sprints são todos os métodos de desenvolvimento não utilizados.

Descrição

No projeto de interfaces, o fato é que seguir etapas previamente definidas, por si só, não exime ninguém do erro. Com efeito, identificar os equívocos mais comuns e saber como evitá-los é algo indispensável no que diz respeito ao sucesso desse tipo de projeto.

Fonte: criação do autor.

Analise as assertivas a seguir.

I. Ignorar o feedback dos usuários pode resultar em uma interface tecnicamente competente, mas difícil de usar.

II. A falta de comunicação clara entre designers e desenvolvedores pode levar a mal-entendidos e implementações errôneas.

III. Negligenciar a acessibilidade evita que se exclua uma parte significativa dos usuários e que se resulte em implicações legais contrárias ao sistema.

Marque a alternativa a seguir que está correta ao indicar quais das assertivas acima são verdadeiras.



A

Somente a I.

☐

B

Somente I e III.

☐

C

Somente a III.

☐

D

Somente I e II.



E

Somente II e III.

☐

Descrição

Qual das frases abaixo é uma vantagem quando se utiliza os padrões de projetos?

**A**

Com o uso de padrões de nomenclatura, o projeto possuirá nomes standardizados, trazendo referência e credibilidade para o software desenvolvido.

**B**

Piora as técnicas e princípios da programação orientada a objetos, como implementação de herança, composição e polimorfismo.

**C**

Aumenta a organização e diminui a manutenção de projetos, já que esses padrões se baseiam em alto acoplamento entre as classes e padronização do código.

**D**

Não implementa hierarquias de herança, diminuindo o tamanho do projeto e aprimorando o mesmo.



10/

<https://aluno.uvv.br/AgendamentoProva/VisualizarCorrecao>**E**

Não possui refatoração de um código, pois é altamente acoplado e de baixa coesão.

Descrição

As noções de separação e independência são fundamentais para o projeto de arquitetura de software porque permitem que alterações sejam localizadas. Com relação aos padrões de arquitetura em camadas (MVC), é INCORRETO afirmar que:



A

no padrão MVC, o componente Controlador gerencia o sistema de dados e as operações associadas a esses dados.



B

o padrão MVC oferece a vantagem de permitir que os dados sejam alterados de forma independente de sua representação e vice-versa.



C

o padrão de arquitetura em camadas, na prática, costuma apresentar a desvantagem de ser difícil proporcionar uma clara separação entre as camadas; e uma camada de alto nível pode ter de interagir diretamente com camadas de baixo nível, em vez de através da camada imediatamente abaixo dela.



D

na arquitetura cliente-servidor, os clientes podem ter de saber os nomes dos servidores disponíveis e os serviços que eles fornecem; no entanto, os servidores não precisam conhecer a identidade dos clientes ou quantos clientes estão acessando seus serviços.



E

no padrão MVC, o componente Model gerencia o sistema de dados e as operações associadas a esses dados.

Descrição

Sobre as vantagens e desvantagens em reutilizar softwares, é correto afirmar que

I-Qualidade do código;

II-Produtividade;

III-Custo maior;

IV-Falta de apoio de ferramenta;

V-Confiabilidade

É correto o que se afirma em::

**A**

I, II e V.

**B**

I, II e III.

**C**

II, IV e V.

**D**

IV e V.

**E**

I, II, III, IV e V

Descrição

São vantagens do uso de framework:

I-Eficiência;

II-Apoio técnico;

III-Dependência;

IV-Códigos desnecessários;

V-Segurança;

É correto o que se afirma em:

**A**

I, II e V

**B**

I, II, III, IV e V

**C**

I, III, e V

**D**

II, III, e IV

**E**

II, IV e V

Descrição

Um sistema pode ser definido como a consequente interação de todos os seus componentes. Sobre os componentes de uma arquitetura de sistema, pode-se dizer que:

**A**

Componentes podem se relacionar com entidades fora dos limites do *software* em questão.

**B**

A interface *requires* define os serviços que são fornecidos pelo componente para outros componentes do sistema.

**C**

A interface *provides* define os serviços que são necessários por um componente para sua correta execução.

**D**

Os componentes devem ser independentes e robustos, e sua comunicação com os demais elementos do sistema pode ser desconsiderada.

**E**

Não é comum que haja colaboração entre componentes, uma vez que estes são entidades independentes.

Leia o texto a seguir:

"A engenharia de software baseada em reuso é uma estratégia da engenharia de software em que o processo de desenvolvimento é orientado para o reuso de softwares existentes. Apesar de o reuso ter sido proposto como uma estratégia de desenvolvimento há mais de 40 anos (McILROY, 1968), só em 2000 o 'desenvolvimento com reuso' se tornou a norma para novos sistemas de negócios... Cada vez mais empresas consideram o software como um ativo valioso. O reuso tem sido promovido para aumentar o retorno sobre os investimentos em software" (Extraído de <https://medium.com/@mikiasoliveira/re%C3%BAso-de-software-8406766d9eb8> em 12/09/2022)

?Sobre a reutilização de software, pode-se afirmar que:



A

De modo geral acaba por piorar a qualidade de software uma vez que o reuso de software implica obrigatoriamente na adoção de tecnologias e códigos defasados.

☐

B

Promove escalabilidade na produção, apesar de ser um dos principais responsáveis pelo aumento de custos.

☐

C

Impede a manutenção dos softwares uma vez que os sistemas são sempre aproveitados como blocos ou caixas-pretas.

☐

D

Promove redução dos custos com impacto negativo na escalabilidade da produção.

☐

E

É um dos principais responsáveis pelo desenvolvimento estruturado de novos softwares, no entanto pode gerar custos na adoção de softwares sem livre licença.

Descrição

O paradigma da arquitetura orientada a serviços (SOA - Service-Oriented Architecture) foi criado com foco em prover serviços fracamente acoplados e coesos como resposta à demanda de integração tecnológica global com diversidade em tecnologias, plataformas e regras de negócio.

Apesar de ser utilizada em diversas aplicações, nos dias atuais esse paradigma ainda não é altamente difundido, isso se deve principalmente porque



A

não existe interesse genuíno das grandes empresas tecnológicas em adotar a SOA e sim em manter estrutura de sistemas orientado a objetos.

☐

B

esse paradigma levaria a um alto custo, pois pressupõe alterar a forma de modelar algo que já está pronto e funcional.



C

não existem vantagens concretas em sua adoção, sendo um paradigma específico para sistemas ineficientes.

☐

D

é muito vantajoso propor uma nova arquitetura para algo que já está em pleno funcionamento.

☐

E

essa tecnologia está em ascensão, sendo ainda muito recente para que ocorra sua completa adoção.

☐

Descrição

No MVC, considere as seguintes propriedades e suas prováveis e respectivas características:

I-gerenciamento de múltiplos visualizadores usando mesmo modelo - **facilidade/dificuldade de manutenção, teste e atualização de sistemas múltiplos;**

II-desenvolvimento em paralelo para o modelo, visualizador e controle - **possível/impossível;**

III-uso em pequenas aplicações - **aconselhável/ desaconselhável** em razão do custo/benefício.

Respectivamente as propriedades I, II e III, as características corretas em negrito são:

**A**

facilidade, possível e desaconselhável.

**B**

facilidade, impossível e aconselhável.

**C**

dificuldade, possível e aconselhável.

**D**

dificuldade, impossível e desaconselhável.

**E**

dificuldade, possível e desaconselhável.



Descrição

Quais são os benefícios para a utilização do MVC no processo de desenvolvimento:

**A**

Segurança, Organização, Eficiência, Tempo, Transformação.

**B**

Insegurança, Organização, Eficiência.

☐**C**

Eficiência, Desorganização, Tempo, Transformação.

☐**D**

Segurança, Ineficiência, Transformação.

☐**E**

Todos acima.

☐

Descrição

O padrão da arquitetura de software MVC (Model-View-Controller) é largamente adotada em diversas aplicações atuais. Com base nessa arquitetura, indique qual é o componente responsável pela apresentação que também recebe os dados de entrada do usuário:

**A**

Data.

☐**B**

Controller.

☐**C**

Application.

☐**D**

View.

**E**

Model

☐

Descrição

Reutilizar componentes é um ato muito comum em desenvolvimento de sistemas. Com relação às vantagens da reutilização de componentes, pode-se afirmar, exceto, que

**A**

Em regra, dificulta a manutenção do software

**B**

Aprimora a produção dos sistemas

☐**C**

Reduz custos operacionais e com pessoal, de modo geral.

☐**D**

Melhora a qualidade do Software.

☐**E**

Podem ser um problema quando não há manutenção de bibliotecas e compatibilidade de APIs.

☐

Descrição

Os padrões de projetos são comumente conhecidos como:



A

Design patterns



B

Patterns design



C

Padronização



D

Metrificação de projetos



E

UX design



Descrição

Sobre o MVC, é correto afirmar que são vantagens:

I-Desenvolvimento simultâneo;

II-Alta coesão;

III-Complexidade;

IV-Dificuldade de modificação;

V-Navegabilidade do código.

É correto o que se afirma em:

**A**

I-II

**B**

I-II-III

**C**

II-III-IV

**D**

II-III

**E**

III-IV



Descrição

Assim como na construção civil, a engenharia de software possui uma subárea denominada Arquitetura. Sobre Arquitetura de Software, é incorreto o que se afirma em:

**A**

A arquitetura de software consiste em um modelo relativamente pequeno e compreensível acerca de como o sistema será estruturado.

☐**B**

A arquitetura de software define a forma como os componentes de software vão se comunicar

☐**C**

A arquitetura de software abrange a estrutura ou as estruturas do sistema e os componentes de software.

☐**D**

A arquitetura de software é uma área independente da engenharia de software.

**E**

A arquitetura de software define as propriedades externamente visíveis dos componentes de software e as relações entre eles.

Descrição

Uma das formas mais importantes de representação de um sistema-alvo é por meio do diagrama de arquétipo estrutural. Com relação a este tópico fundamental, assinale a alternativa incorreta:



A

Trata-se de um requisito essencial para o usuário final do sistema-alvo, sem o qual o mesmo não seria capaz de utilizá-lo.



B

O arquétipo representa o sistema de interesse em uma linguagem de alto nível.



C

É componente fundamental dessa representação o barramento ou interface de comunicação.



D

É de fácil compreensão para todos os envolvidos no desenvolvimento do sistema.



E

Sua implementação é uma etapa fundamental para desenvolvimento de projetos de arquitetura de sistemas.

Descrição

Sobre as vantagens e desvantagens em reutilizar softwares, é correto afirmar que

I-Qualidade do código;

II-Produtividade;

III-Custo maior;

IV-Falta de apoio de ferramenta;

V-Confiabilidade

É correto o que se afirma em::

**A**

I, II e V.

**B**

I, II e III.

**C**

II, IV e V.

**D**

IV e V.

**E**

I, II, III, IV e V



Descrição

A respeito do MVC, assinale a alternativa correta:

**A**

A comunicação entre as interfaces e as regras de negócio é responsabilidade da camada controller.

**B**

Trata-se de um modelo conhecido por dificultar a documentação.

☐**C**

A camada view é responsável pelas regras de negócio.

☐**D**

A maior desvantagem desse padrão é não ser aplicável a muitos contextos, o que diminui o reuso de código.

☐**E**

A camada model é responsável pela exibição de dados na tela.

☐

Descrição

O que são instâncias dentro do contexto de arquétipos estruturais em arquitetura de sistemas:

**A**

são métodos executáveis de programação para acelerar o processo.

**B**

são representações reais das componentes de um sistema.

**C**

são funções não implementadas (não reais) de um componente.

**D**

são elementos geralmente empregados para a construção de sistemas.

**E**

são outra forma de denominar componentes.

Descrição

Quais são os benefícios para a utilização do MVC no processo de desenvolvimento:

**A**

Segurança, Organização, Eficiência, Tempo, Transformação.

**B**

Insegurança, Organização, Eficiência.

**C**

Eficiência, Desorganização, Tempo, Transformação.

**D**

Segurança, Ineficiência, Transformação.

**E**

Todos acima.

Descrição

Sobre a engenharia de software baseada em componentes, quais são algumas das características essenciais de um componente?



A

Padronizado, Independente, Passível de composição, implantável, Documentado



B

Padronizado, Bonito, Testado, Implantável



C

Padronizado, Dependente, Implantável, Simples



D

uno.uv.br/AgendamentoProva/VisualizarCorrecaoAluno/17453520222?IdDisciplinaOferta=130041&token=rYDRvGphidelisbXSw%3D

22 06:00

Portal do Aluno

Implantável, Documentado, Simples, Testado, Padronizado



E

Todos acima